Efficient LLM Alignment for STEM Education: DPO, MCQA, Quantization, and RAG

Clémentine Jordan | 347046 | clementine.jordan@epfl.ch
Shuli Jia | 316620 | shuli.jia@epfl.ch
Kyuhee Kim | 395557 | kyuhee.kim@epfl.ch
Stergios Konstantinidis | 413018 | stergios.konstantinidis@unil.ch
MostlyNaturalLanguagePirates

Abstract

General-purpose large language models (LLMs) often underperform on domain-specific tasks such as university-level STEM coursework, which require structured reasoning and factual precision. We present a modular adaptation of `Qwen3-0.6B-Base` into a suite of lightweight academic assistants for EPFL education. The system comprises four specialized models: a multiple-choice QA model with chain-of-thought supervision, a DPO model stabilized via balanced initialization and preference filtering, a RAG model with contrastive retrieval and fine-tuned generation, and a quantized variant for efficient deployment. Each module contributes measurable improvements: DPO (+11.3), MCQA (+7.9), RAG (+8.2), and quantization reduces memory by 65% with minimal degradation. These results underscore the effectiveness of modular, task-specific adaptation for academic LLM use.

1 Introduction

Large Language Models (LLMs) perform well on general tasks but often underperform in domain-specific contexts like university-level STEM education, where structured reasoning and factual precision are essential. This gap stems from the misalignment between generic pretraining and the demands of complex academic problem solving.

We adapt `Qwen3-0.6B-Base` into a compact academic assistant for EPFL coursework via a modular pipeline with four components: (1) a multiple-choice QA (MCQA) model using chain-of-thought (CoT) supervision (Wei et al., 2022) to enhance reasoning; (2) Direct Preference Optimization (DPO) (Rafailov et al., 2024), stabilized through balanced initialization (Feng et al., 2024) and filtered preferences (Morimura et al., 2024); (3) a RAG module with contrastive retrieval and fine-tuned generation; and (4) quantization comparing weight-only and weight–activation strategies for efficient deployment.

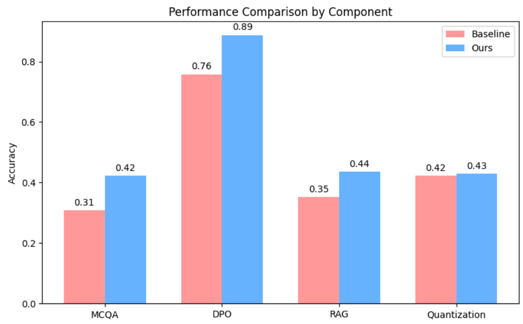

Figure 1: Paired comparison between the baseline and our model across four components. Each configuration from Section 3.2 is evaluated on the primary benchmark in Section 3.3.

Figure 1 summarizes the performance gains from each module, demonstrating the effectiveness of our targeted adaptation strategy.

2 Approach

DPO We use DPO (Rafailov et al., 2024) to align `Qwen3-0.6B` with human preferences, optimizing : $\mathcal{L}_{DPO} = -\log\sigma\big(\beta\big[\log\pi_\theta(y_\text{w}|x) - \log\pi_\theta(y_\text{l}|x) - \log\pi_\text{ref}(y_\text{w}|x) + \log\pi_\text{ref}(y_\text{l}|x)\big]\big)$, where π_θ is the trainable policy, π_ref the frozen reference, and β controls preference sharpness.

While the core DPO framework is fixed, its effectiveness depends heavily on implementation choices. We observed improved alignment by (1) initializing with supervised fine-tuning (SFT) on a balanced, STEM-centric dataset (balanced), (2) training on a rigorously filtered and aggregated preference dataset (`default` combination), and (3) setting $\beta = 0.005$ to promote gradient stability. A pre-DPO phase—using a DPO-trained model for guidance (Pan et al., 2025)—was evaluated but showed no consistent benefit and was excluded from the final model.

MCQA We fine-tuned `Qwen3-0.6B-Base` using HuggingFace `trl`'s `SFTTrainer` to adapt it for supervised STEM MCQA. The training optimizes the causal language modeling loss: $\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$ where x is the input token sequence and θ the model parameters. Our main contributions include: (1) implementing a CoT by generating rationales for the Milestone 1 dataset using GPT-4o-mini and evaluating their impact, (2) selecting and combining public datasets based on individual and joint performance, (3) tuning training parameters for stability and efficiency, and (4) experimenting with prompt designs. The best results were achieved using CoT on a combination of OpenBookQA (Mihaylov et al., 2018) and Milestone 1 dataset.

Quantization To reduce inference memory usage while maintaining model performance, we explored two quantization strategies: weight-only quantization using `BitsAndBytes` and joint weight–activation quantization using `LLMCompressor`. Formally, given a pretrained model with weight matrices $W \in \mathbb{R}^{m \times n}$, weight-only quantization maps W to low-precision representations $\hat{W} \in \mathbb{Q}^{m \times n}$, typically in 8-bit format. Our experiments showed that the 8-bit `BitsAndBytes` variant yielded the best trade-off, significantly reducing memory consumption while preserving performance.

RAG Our RAG system adopts a dual-stage architecture comprising a contrastively trained sentence-transformer retriever and a generator fine-tuned from `Qwen3-0.6B-Base`. Cosine similarity was chosen after evaluating multiple scoring functions. The retriever was optimized using `ContrastiveTensionDataLoader`, while the generator was fine-tuned with `SFTTrainer` on a filtered subset of the RAG dataset initialized from the MCQA-tuned model.

3 Experiments

3.1 Data

All training datasets were validated—through both automated and manual checks—to ensure no overlap with evaluation benchmarks or banned benchmark training sets.

DPO We constructed our DPO training datasets using `M1` and public preference sources. Of the four filtered `M1` variants, we selected `v2` for its clear overall preferences (A/B only) and internally consistent secondary fields (AB allowed). As shown in Milestone 2, `v2` provided a strong balance between scale and label quality and served as the primary `M1` source (see Appendix B).

We also included UltraFeedback (Cui et al., 2023), Code-Preference-Pairs (Vezora, 2024), SHP (Ethayarajh et al., 2022), and Math-Step-DPO-10K (Lai et al., 2024), each formatted as `(prompt, chosen, rejected)`. SHP was excluded after individual evaluation due to low utility (Appendix C). Table 1 summarizes all combinations.

Dataset	**Size**	**Default**	**Math+**
`M1-v2`	19.7K	✓	✓
`UltraFeedback`	61.1K	✓	✓
`Code-Preference-Pairs`	54K	✓	✓
`Math-Step-DPO-10K`	10.8K	–	✓
Total	–	135K	146K

Table 1: DPO training dataset configurations.

We additionally applied SFT before DPO, using instruction–output pairs from SciQ (Johannes Welbl, 2017), OpenBookQA, MBPP (Austin et al., 2021), MathInstruct (Yue et al., 2023), and HendrycksMath (Hendrycks et al., 2021), formatted as single-input sequences by concatenating instruction and output. To assess domain impact, we created SFT variants via source subsampling (Table 2).

Variant	**SciQ**	**OpenBookQA**	**MBPP**	**Math-Instruct**	**Hendrycks-Math**	**Total**
`lightweight`	5K	–	374	10K	–	15.4K
`balanced`	10K	5K	374	20K	7.5K	42.8K
`math_only`	–	–	–	20K	7.5K	27.5K
`reasoning`	10K	5K	–	–	–	15K
`curriculum`	4K	4K	374	16K	7.5K	31.8K

Table 2: SFT variant dataset composition

MCQA We constructed our MCQA training dataset by combining the `M1` MCQA dataset with public datasets, including SciQ, OpenBookQA, and QASC (Khot et al., 2020), as outlined in Milestone 2. To broaden STEM coverage, we added MedMCQA (Pal et al., 2022) and MathQA (Amini et al., 2019), each evaluated via standalone SFT (see Appendix E); MedMCQA was later excluded due to misalignment and lower accuracy.

For reasoning supervision, we applied CoT prompting. Rationales for `M1` were generated using a GPT-4o-mini pipeline, while public datasets were evaluated with both original and regenerated rationales (see Appendices D,E). Based on performance (AppendixE), we selected CoT-augmented combinations of `M1`, OpenBookQA, and SciQ, shown in

Table 3. Each instance followed the format: `<Q>` A. `[A]` B. `[B]` C. `[C]` D. `[D]` [Reasoning: <...>] Answer: `<Label>`.

Dataset	Size	M1+OB	M1+SQ	OB+SQ	All
M1 (generated CoT)	789	✓	✓	–	✓
OpenBookQA (original CoT)	4.96K	✓	–	✓	✓
SciQ (original CoT)	11.7K	–	✓	✓	✓
Total Size	–	5.75K	12.5K	16.66K	17.45K

Table 3: Evaluated CoT-augmented dataset configurations.

Quantization For quantization experiments, we used subsets of the M1 MCQA dataset ($n = 789$, without CoT), as well as a combined M1+OB dataset (without CoT), comprising M1 and OpenBookQA samples. For the M1 dataset, calibration set sizes of 128, 512, and 789 were evaluated; for M1+OB, sizes of 128, 512, and 1024 were used. Calibration datasets were randomly sampled from the respective collections.

RAG We constructed the RAG retrieval corpus from a diverse mix of QA and STEM-related datasets. To satisfy the 100K-document and 512-token-per-document constraints, we first excluded all entries exceeding the token limit. For the remaining documents, we applied a diversity filter based on pairwise cosine similarity, retaining the least similar segments to maximize topical coverage. For long arXiv papers, we used GPT-4o to generate concise summaries (see Appendix F).

In addition, we created an SFT subset for fine-tuning the generation model by selecting the 1,000 least-redundant documents across datasets, ranked by average distance. Table 4 summarizes the retrieval and SFT dataset sizes after filtering.

Dataset	Retrieval Size	SFT Size
M1 Preferences	1,082	132
Natural Questions (Kwiatkowski et al., 2019)	66,148	389
KILT Tasks (Petroni et al., 2021)	2,788	410
FineTome-100k[1]	26,176	60
arXiv Recaps (GPT-4o) (OpenAI et al., 2024)	3,804	9

Table 4: Filtered datasets used for RAG retrieval and additional SFT.

3.2 Experimental details

DPO We conducted all experiments using HuggingFace's `DPOTrainer` with `Qwen3-0.6B-Base` as the policy model. To assess DPO sensitivity, we varied (i) SFT initialization, (ii) preference dataset type, (iii) optimization hyperparameters, and (iv) the use of Pre-DPO.

SFT variants included five instruction-tuning datasets and a no-SFT baseline. Two types of preference datasets were used: `Default` and `+Math`. Optimization settings covered a base configuration (β=0.1, LR=5e-6), an aggressive setup (higher LR, more epochs), and variants inspired by Pan et al. (2025); Meng et al. (2024) with lower β and longer inputs. We also evaluated Pre-DPO under its original and modified configurations.

Unless otherwise noted, models were trained for one epoch with fp16, batch size 2 (grad. acc. = 4), and max input length 512. **The baseline** is the DPO model trained on `default` preference dataset without SFT using the base dpo configuration. Table 5 summarizes all setups.

Category	Variants
SFT Initialization	`balanced`, `math_only`, `lightweight`, `reasoning`, `curriculum`, None (no SFT)
Preference Dataset	`default`, `math+`
DPO Configs	• `base`: $\beta = 0.1$, LR = 5e−6 • `aggressive`: $\beta = 0.1$, LR = 2e−5, 3 epochs • `suggested`: $\beta = 0.005$, LR = 1e−6, len = 1024 • `lower_beta`: $\beta = 0.005$, LR = 5e−6 • `slight_lower_beta`: $\beta = 0.05$, LR = 5e−6
Pre-DPO Configs	• `default`: $\beta = 0.05$, LR = 5e−6 • `lower_beta`: $\beta = 0.01$, LR = 5e−6 • `aggressive`: $\beta = 0.2$, LR = 8e−7 • None (no Pre-DPO)

Table 5: Summary of experimental variants across SFT initialization, preference dataset composition, and training configurations for both DPO and Pre-DPO.

MCQA We fine-tuned all models using HuggingFace's `trl` `SFTTrainer` with FP16 precision. The final setup used a learning rate of 1×10^{-5}, 3 epochs, batch size 1 with gradient accumulation of 8, and a 512-token length. Higher learning rates or shorter training led to underfitting, whereas five epochs resulted in overfitting (Appendix G).

For models trained with CoT reasoning, we extended inputs to include rationales. Additionally, we experimented with a domain-specific prompt (`new-prompt`): *"The following are multiple choice questions (with answer) about knowledge and skills in advanced master-level STEM courses,"* appended before the question. **The baseline** is `Qwen3-0.6B-Base` without any fine-tuning.

Quantization Table 6 summarizes the quantization configurations. We performed PTQ on two

MCQA models (M1-only and best-performing), using BitsAndBytes (W4, W8) for weight-only quantization and LLMCompressor (W8A8, W4A16) for full quantization via GPTQ and SmoothQuant. Two calibration datasets (`M1` and `M1+OB`) were used with varying sizes. For SmoothQuant, we tuned the `smoothing_alpha` parameter (0.1–1.0) to control the trade-off between activation and weight scaling. The full-precision best-performing MCQA model serves as the **baseline**.

Config	Options
Quantization Method	`BNB` - W8, W4 `LLMComp` (GPTQ + SmoothQuant) - W8A8, W4A16
Base Model	`M1-only` (MCQA model trained on `M1` only) `Best` (Best-performing MCQA model)
Calibration Dataset	`M1` - 128, 512, 789 `M1+OB` - 128, 512, 1024
Smoothing Alpha	0.1, 0.3, 0.5, 0.8, 1.0

Table 6: Quantization experiment configurations. BNB = BitsAndBytes; LLMComp = LLMCompressor.

RAG We optimized our RAG setup across four components: similarity function, retrieval depth, embedding model, and generation model.

A preliminary search over similarity functions and retrieval depths showed cosine and $k = 10$ as ideal (Appendix J). We reused them for all testing.

For embedding, we compared the baseline `E5-STEM` with task-specific variants (`M3+Neg v1–v3`) trained with contrastive learning with batch size 20, 20 and 21 and pos/neg ratios of 1, 2, and 3.

For generation, we used the `MCQA-Baseline`[2], `Best-Performing MCQA model` and a `RAG-SFT` variant fine-tuned on the RAG subset (see Section 3.1) using identical SFT hyperparameters: lr 1×10^{-5}, 3 epochs, grad 8, seq len 512, batch size 1. Table 7 summarizes all configurations. **Baselines** include: (1) `sucharush/e5_stem_finetuned` for embedding with MCQA generation; (2) `Qwen 3-0.6B` for generation with `M3+Neg v1` embedding.

	Config	Options
Preliminary	**Similarity Function**	Cosine, Dot Product, Max Inner Product, Jaccard
	Retrieval Depth (top-k)	5, 10, 20
Main	**Embedding Model**	`E5-STEM` (Baseline; sucharush/e5_stem_finetuned) `M3+Neg v1` — pos_neg_ratio = 1 `M3+Neg v2` — pos_neg_ratio = 2 `M3+Neg v3` — pos_neg_ratio = 3
	Generation Model	`MCQA Baseline` (pre-trained on M1 with CoT)[3] `Best-Performing MCQA model` + `RAG-Finetuned` (further tuned with RAG-SFT dataset)

Table 7: RAG experiment configurations.

[2]`shulijia/MNLP_M3_mcqa_model_simpleVal_m1_cot`, trained on 90% of M1 with CoT

3.3 Evaluation Method

DPO We evaluated model performance using pairwise accuracy. The primary evaluation was conducted on `zechen-nlp/MNLP_dpo_evals` ($n = 528$)[4]. To broaden the evaluation, we also converted several public datasets to the unified `{prompt, chosen, rejected}` format[5], as summarized in Table 8.

Original Dataset	Size	Config	Conversion Summary
MT-Bench (Zheng et al., 2023)	1.28K 882	`mtbench-human` `mtbench-gpt4`	Used last-turn prompt and responses; filtered to matching instructions.
HH-RLHF (Bai et al., 2022)	8.53K	`hh-dpo-eval`	Used last assistant reply; only exact prompt matches.
SHP (Bai et al., 2022)	18.4K	`shp-dpo-eval`	Used full history as prompt; filtered by valid label.

HelpSteer3 (Wang et al., 2025)	915
	432
	243
		`helpsteer3-general`
		`helpsteer3-code`
		`helpsteer3-stem`
			Filtered by non-neutral preference; prompt is concatenated user turns.

Table 8: Summary of public evaluation datasets converted to `LightEval` format. Test or validation splits only.

MCQA We evaluate accuracy on multiple-choice questions with exactly four options and one correct answer, using the evaluation sets shown in Table 9. Only standardized or filtered questions with the matching format are included. `mnlp_stem_mcqa` set serves as the primary benchmark.

Config	**Original Dataset**	**Size**
`mnlp_stem_mcqa`	MNLP_STEM_mcqa[6]	770
`mmlu`	MMLU (Hendrycks et al., 2020)	14,042
`nlp4ed`	NLP4Education (Borges et al., 2024)	2,147
`ai2arc`	AI2_ARC (Clark et al., 2018)	3,519

Table 9: MCQA evaluation datasets. All samples have four answer options and one correct answer.

Quantized Quantized models were evaluated on the same benchmark datasets as the MCQA baseline. Peak VRAM usage was measured across ten runs of a simple STEM MCQA example. We considered accuracy, as well as an efficiency metric defined by the ratio accuracy/peak VRAM usage.

RAG To assess the impact of retrieval augmentation, we evaluated the RAG model on the same MCQA evaluation datasets. The evaluation sets were limited up to 1,000 samples to avoid memory issues and ensure efficiency.

[4]`https://huggingface.co/datasets/zechen-nlp/MNLP_dpo_evals`

[5]`https://huggingface.co/collections/koreankiwi99/epfl-lighteval-dpo-datasets-68474bc94137e41ade8e560f`

#	SFT Data	DPO Data	DPO Config	Pre-DPO	mnlp-dpo-evals		shp-dpo-eval		hh-dpo-eval		helpsteer3-code		helpsteer3-stem		helpsteer3-general		mtbench-gpt4		mtbench-human	
					Acc	StdErr	Acc	StdErr	Acc	StdErr	Acc	StdErr	Acc	StdErr	Acc	StdErr	Acc	StdErr	Acc	StdErr
0	**no-SFT (Baseline)**	default	base	-	0.758	0.019	0.453	0.004	0.518	0.005	0.507	0.024	0.494	0.032	0.522	0.017	0.677	0.016	0.636	0.013
1	no-SFT	default	lower_beta	-	0.831	0.016	**0.461**	0.004	0.510	0.005	0.532	0.024	0.621	0.031	0.548	0.016	0.622	0.016	0.543	0.014
2	no-SFT	default	slight_lower_beta	-	0.782	0.018	0.456	0.004	**0.520**	0.005	0.530	0.024	0.490	0.032	0.522	0.017	0.679	0.016	**0.640**	0.013
3	no-SFT	default	suggested	-	0.807	0.017	0.447	0.004	0.511	0.005	0.620	0.023	0.646	0.031	**0.573**	0.016	0.605	0.016	0.528	0.014
4	no-SFT	default	aggressive	-	0.676	0.020	0.419	0.004	0.514	0.005	0.461	0.024	0.543	0.032	0.463	0.016	0.535	0.017	0.515	0.014
5	no-SFT	math+	base	-	0.735	0.019	0.439	0.004	0.519	0.005	0.491	0.024	0.490	0.032	0.511	0.017	**0.694**	0.016	**0.640**	0.013
6	no-SFT	math+	lower_beta	-	0.792	0.018	0.430	0.004	0.511	0.005	0.537	0.024	0.597	0.032	0.545	0.016	0.638	0.016	0.567	0.014
7	lightweight	default	lower_beta	-	0.835	0.016	0.433	0.004	0.512	0.005	0.560	0.024	0.626	0.031	0.550	0.016	0.613	0.016	0.543	0.014
8	reasoning	default	lower_beta	-	0.843	0.016	0.423	0.004	0.510	0.005	**0.567**	0.024	0.638	0.031	0.554	0.016	0.616	0.016	0.537	0.014
9	math_only	default	lower_beta	-	0.854	0.015	0.442	0.004	0.508	0.005	0.560	0.024	**0.650**	0.031	0.540	0.016	0.616	0.016	0.543	0.014
10	curriculum	default	lower_beta	-	0.858	0.015	0.425	0.004	0.508	0.005	0.549	0.024	0.617	0.031	0.543	0.016	0.605	0.016	0.533	0.014
11	**balanced (Ours)**	default	balanced	-	**0.888**	0.014	0.440	0.004	0.506	0.005	0.546	0.024	0.605	0.031	0.536	0.016	0.598	0.017	0.521	0.014
12	balanced	default	lower_beta	lower_beta	0.864	0.015	0.443	0.004	0.512	0.005	0.553	0.024	0.609	0.031	0.544	0.016	0.603	0.016	0.541	0.014
13	balanced	default	lower_beta	aggressive	0.841	0.016	0.436	0.004	0.510	0.005	0.562	0.024	0.613	0.031	0.558	0.016	0.552	0.017	0.516	0.014
14	curriculum	default	lower_beta	base	0.852	0.015	0.428	0.004	0.511	0.005	0.539	0.024	0.609	0.031	0.556	0.016	0.607	0.016	0.531	0.014
15	lightweight	default	lower_beta	base	0.826	0.017	0.443	0.004	0.517	0.005	0.549	0.024	0.601	0.031	0.542	0.016	0.611	0.016	0.544	0.014

Table 10: Evaluation results across different SFT and DPO configurations. Accuracy (Acc) and Standard Error (StdErr) are reported for each evaluation category.

3.4 Results

DPO Table 10 shows that lowering the DPO beta alone (row 1) improves main accuracy from 0.758 (baseline) to 0.831. The best overall performance comes from the balanced (Ours) setup, achieving 0.888 on the main evaluation and strong results across `helpsteer3` and `mtbench`. Notably, `math_only` yields the highest `helpsteer3-STEM` score (0.650), and `reasoning` leads on `helpsteer3-code` (0.567).

MCQA Table 11 summarizes the accuracy of fine-tuned MCQA models across four benchmarks. In preliminary experiments, SFT on the aggregated datasets (excluding MedQA) improved performance (e.g., 0.3078 to 0.3948 on `mnlp_stem_mcqa`), but adding CoT reduced it (e.g., 0.3857), prompting a closer look at CoT quality.

Method	Dataset	mnlp_stem_mcqa	mmlu	nlp4edu	ai2arc
Qwen3-0.6B-Base (Baseline)	–	0.3078	0.3834	0.3810	0.5558
Preliminary Research					
SFT	Full	0.3948	0.4495	0.4569	0.7045
SFT + CoT	Full	<u>0.3857</u>	0.4557	<u>0.4457</u>	<u>0.6885</u>
Effect of Domain-specific Prompt					
SFT + CoT	M1	0.4104	0.4726	0.4499	0.6974
SFT + CoT + new prompt	M1	0.4026	0.4700	0.4565	0.6945
CoT Combination					
SFT + CoT	OB	0.4130	0.4599	0.4341	0.6982
	SciQ	0.4169	**0.4778**	0.4327	0.7133
	OB+SQ	0.4052	0.4776	0.4523	0.7121
	M1+SQ	0.4156	0.4731	0.4783	0.7059
	M1+OB (Ours)	**0.4221**	0.4702	**0.4905**	0.6968
	All	0.4052	0.4777	0.4872	**0.7164**
Standard error	–	± 0.0166	± 0.0041	± 0.0105	± 0.0084

Table 11: Accuracy of MCQA models fine-tuned with different configurations across four benchmarks. "Full" denotes OpenBookQA + SciQ + QASC + MathQA (53.12K). Bold = best; underline = notable results.

A model trained on just 789 CoT examples from `M1` outperformed the full 53.12K CoT setting (0.4104 vs. 0.3857), showing that quality outweighs quantity. We compared official and revised CoTs (Appendix **??**) and found notable variance. Task-specific prompts showed no consistent gains.

Combining top CoT datasets, our final model (`M1+OB`) achieved the best results on `mnlp_stem_mcqa` (0.4221) and `nlp4edu` (0.4905), while `All` led on `ai2arc` (0.7164), highlighting the importance of careful CoT selection.

Quantized Table 12 shows that the best accuracy on the `mnlp_stem_mcqa` benchmark comes from the `LLMComp` W8A8 configuration applied to the best MCQA model. However, BitsAndBytes (BnB) 8-bit offers a more practical trade-off: it achieves near full-precision accuracy (0.4156) while reducing VRAM usage from 2.4 GB to 0.8 GB. Although BnB 4-bit has the highest accuracy-to-VRAM ratio (0.57), its low raw accuracy (0.3429) makes it less useful in practice. The W4A16 setup consistently underperforms in both accuracy and efficiency. Overall, BnB 8-bit emerges as the best quantization strategy for maintaining performance while improving resource efficiency.

Model	Library	Config	Samples	Smooth	Acc	Acc norm.	VRAM (GB)	Acc / VRAM
M1-only	LLMComp	W8A8	1024	0.8	0.4286	0.3351	1.2	0.3572
		W4A16	1024	0.3	0.4052	0.3078	1.4	0.2894
	BNB	W8	–	–	0.4130	0.3325	0.8	0.5163
		W4	–	–	0.3429	0.2831	0.6	0.5715
Best	LLMComp	W8A8	512	0.5	0.4234	0.3831	1.2	0.3528
		W4A16	1024	0.8	0.3870	0.3610	1.4	0.2764
	BNB	W8	–	–	0.4156	0.3714	0.8	0.5195
		W4	–	–	0.3260	0.3104	0.6	0.5433

Table 12: Performance on `mnlp_stem_mcqa` and VRAM usage across quantized MCQA configurations. The full-precision baseline achieves 0.4221 accuracy. Acc/VRAM column reflects raw accuracy per GB of memory.

As presented in the full results in the Appendix H, variations in smoothing strength or calibration size had minimal effect on accuracy. Results fluctuate slightly across benchmarks, but relative performance between models remains consistent, as shown in Appendix I.

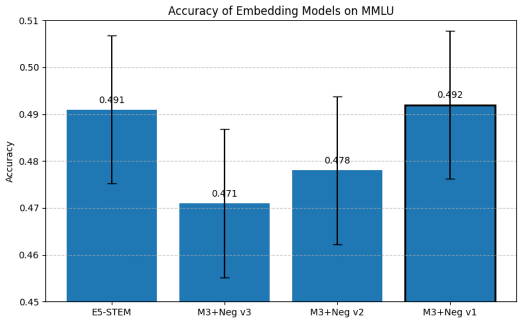

Figure 2: Accuracy of different embedding models our `mmlu` evaluation subset

RAG Figure 2 shows that the `M3+Neg v1` embedding achieved the highest accuracy (0.492 ± 0.0158) on `mmlu`. Using this encoder, Table 13 summarizes accuracy by generation model across all datasets. RAG-SFT consistently outperforms the Qwen 3-0.6B baseline.

Evaluation Data	Model	Accuracy	Std
`ai2arc`	Qwen 3-0.6B (Baseline)	0.641	± 0.0152
	RAG-SFT variant (Ours)	**0.7195**	± 0.0076
`nlp4ed`	Qwen 3-0.6B (Baseline)	0.346	± 0.0151
	RAG-SFT variant (Ours)	**0.4628**	± 0.0113
`mmlu`	Qwen 3-0.6B (Baseline)	0.400	± 0.0155
	MCQA-Baseline	0.492	± 0.0158
	Best-Performing MCQA model	0.488	± 0.0158
	RAG-SFT variant (Ours)	**0.494**	± 0.0158
`mnlp_stem_mcqa`	Qwen 3-0.6B (Baseline)	0.3532	± 0.0178
	Best-Performing MCQA model	0.4234	± 0.0178
	RAG-SFT variant (Ours)	**0.4351**	± 0.0179

Table 13: Comparison of model accuracies (± std) for each evaluation dataset. Baseline refers to Qwen 3-0.6B; RAG-SFT (Ours) denotes our proposed method. Bolded numbers indicate the highest accuracy per dataset.

4 Analysis

DPO To better understand system limitations, we analyzed examples that failed across all DPO models (Appendix K). Frequent failures occurred in math word problems (e.g., Example98) and code generation tasks across Rust, Go, and JavaScript (e.g., Examples294, 274, 279). These cases expose challenges in domain-specific reasoning and structured generation, suggesting directions for future data augmentation or targeted fine-tuning.

MCQA CoT prompting improved performance, but combining datasets often reduced accuracy—likely due to style or reasoning mismatches. This suggests that data quality and alignment matter more than quantity. More explicit prompts did not yield clear gains.

Quantized Models Although BnB 4-bit had the best accuracy-to-VRAM ratio, its low raw accuracy makes it less practical. BnB 8-bit provided a better balance of performance and efficiency.

RAG RAG integration improved MCQA accuracy, but results varied widely across hyperparameters (Appendix J). Further tuning and evaluation iterations are needed to maximize performance.

5 Ethical considerations

Multilingual Adaptation Though trained on English data, our modular setup supports multilingual extension. MCQA can incorporate datasets like MLQA and MKQA (Lewis et al., 2020; Longpre et al., 2021), and English reward models can pseudo-label data cross-lingually (Yang et al., 2025). However, quantization may degrade performance in non-Latin scripts and STEM tasks (Marchisio et al., 2024), and low-resource adaptation requires culturally relevant data often missing today.

Potential Benefits and Harms If effective, the model aids STEM education, especially for learners and teachers. Risks include reinforcing bias, curriculum mismatch, over-reliance, or misuse for cheating and misinformation.

Harm Mitigation Strategies Curating diverse datasets, adopting community-based evaluations, and using transparency tools and moderation can reduce misuse and bias.

Impact on Marginalized Groups Without inclusive training data, marginalized users may receive less accurate or relevant responses. Addressing this needs participatory design and culturally aware data curation.

6 Conclusion

This project explores methods to improve alignment, reasoning, and efficiency in language models. We applied DPO, rationale prompting, retrieval training, and quantization, each showing gains in their respective areas. Limitations include narrow configuration coverage and reliance on automatic metrics. Future work should enhance CoT reliability, adopt broader human-centered evaluation, and explore joint optimization across components for more integrated improvements.

References

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. MathQA: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367, Minneapolis, Minnesota. Association for Computational Linguistics.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback.

Beatriz Borges, Negar Foroutan, Deniz Bayazit, Anna Sotnikova, Syrielle Montariol, Tanya Nazaretzky, Mohammadreza Banaei, Alireza Sakhaeirad, Philippe Servant, Seyed Parsa Neshaei, Jibril Frej, Angelika Romanou, Gail Weiss, Sepideh Mamooler, Zeming Chen, Simin Fan, Silin Gao, Mete Ismayilzada, Debjit Paul, Philippe Schwaller, Sacha Friedli, Patrick Jermann, Tanja Käser, Antoine Bosselut, EPFL Grader Consortium, and EPFL Data Consortium. 2024. Could chatgpt get an engineering degree? evaluating higher education vulnerability to ai assistants. *Proceedings of the National Academy of Sciences*, 121(49):e2414955121.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the AI2 reasoning challenge. *CoRR*, abs/1803.05457.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. 2023. Ultrafeedback: Boosting language models with high-quality feedback.

Kawin Ethayarajh, Yejin Choi, and Swabha Swayamdipta. 2022. Understanding dataset difficulty with $\mathcal{V}$-usable information. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5988–6008. PMLR.

Duanyu Feng, Bowen Qin, Chen Huang, Zheng Zhang, and Wenqiang Lei. 2024. Towards analyzing and understanding the limitations of dpo: A theoretical perspective.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *CoRR*, abs/2009.03300.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *NeurIPS*.

Matt Gardner Johannes Welbl, Nelson F. Liu. 2017. Crowdsourcing multiple choice science questions.

Tushar Khot, Peter Clark, Michal Guerquin, Peter Jansen, and Ashish Sabharwal. 2020. Qasc: A dataset for question answering via sentence composition. *arXiv:1910.11473v2*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics*, 7:452–466.

Xin Lai, Zhuotao Tian, Yukang Chen, Senqiao Yang, Xiangru Peng, and Jiaya Jia. 2024. Step-dpo: Step-wise preference optimization for long-chain reasoning of llms. *arXiv preprint arXiv:2406.18629*.

Patrick Lewis, Barlas Oguz, Ruty Rinott, Sebastian Riedel, and Holger Schwenk. 2020. MLQA: Evaluating cross-lingual extractive question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7315–7330, Online. Association for Computational Linguistics.

Shayne Longpre, Yi Lu, and Joachim Daiber. 2021. MKQA: A linguistically diverse benchmark for multilingual open domain question answering. *Transactions of the Association for Computational Linguistics*, 9:1389–1406.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. 2024. How does quantization affect multilingual llms?

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward.

Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *EMNLP*.

Tetsuro Morimura, Mitsuki Sakamoto, Yuu Jinnai, Kenshi Abe, and Kaito Ariu. 2024. Filtered direct preference optimization.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. Gpt-4 technical report.

Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR.

Junshu Pan, Wei Shen, Shulin Huang, Qiji Zhou, and Yue Zhang. 2025. Pre-dpo: Improving data utilization in direct preference optimization using a guiding reference model.

Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: a Benchmark for Knowledge Intensive Language Tasks. ArXiv:2009.02252.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model.

Vezora. 2024. Code-preference-pairs. `https://huggingface.co/datasets/Vezora/Code-Preference-Pairs`. Accessed: 2025-05-25.

Zhilin Wang, Jiaqi Zeng, Olivier Delalleau, Hoo-Chang Shin, Felipe Soares, Alexander Bukharin, Ellie Evans, Yi Dong, and Oleksii Kuchaiev. 2025. Helpsteer3-preference: Open human-annotated preference data across diverse tasks and languages.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. 2022. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903.

Wen Yang, Junhong Wu, Chen Wang, Chengqing Zong, and Jiajun Zhang. 2025. Implicit cross-lingual rewarding for efficient multilingual preference alignment.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena.

A AI Usage

We used GPT-4o and GPT-4o-mini to generate MCQA rationales and to recap ArXiv papers for RAG. Additionally, we used ChatGPT to assist with grammar checking and LaTeX formatting. All core writing and contributions remain original and authored by us.

B DPO Assessment of Milestone 1 Dataset Variants

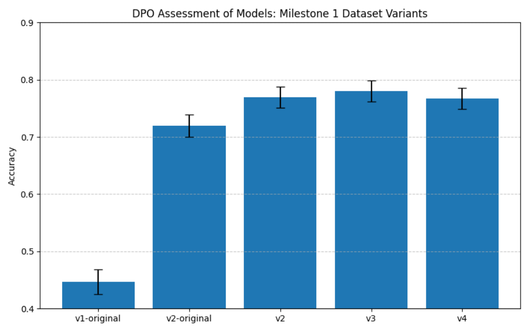

Figure 3: Accuracy of DPO-trained models using Milestone 1 dataset variants, evaluated on the `mnlp-dpo-eval` set. All models were trained in Milestone 2. The `v2-original` model follows the original configuration (learning rate 1×10^{-5}, no gradient clipping), while all others use the tuned setup (learning rate 5×10^{-6}, `max_grad_norm` = 1.0).

C DPO Assessment of Models: Public DPO Datasets

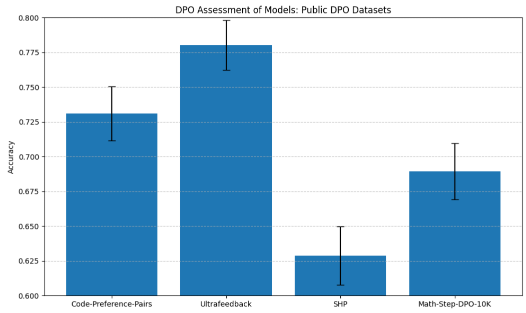

Figure 4: Accuracy of DPO-trained models using public preference datasets, evaluated on the `mnlp-dpo-eval` set. All models were trained using the tuned setup (learning rate 5×10^{-6}, `max_grad_norm` = 1.0).

D Rationale Prompting Method for MCQA Model

To generate high-quality rationales, we used the GPT wrapper provided by the course, initiating a separate chat session for each question. A consistent prompt was refined through trial runs on a small EPFL subset and selected based on manual inspection for clarity and technical soundness.

The prompt asked the model to explain why the correct answer is valid without simply restating it. The final prompt was:

*"You are given a multiple-choice question and its correct answer(s). Your task is to provide a clear, technically sound rationale explaining *why* the correct choice(s) are correct — without restating the answer verbatim. The explanation should reflect the knowledge and reasoning skills expected from students in advanced, master-level STEM courses. Question:* `question`*. Choices:* `options`*. Correct Answer(s):* `answer`*. Now provide the rationale. Avoid starting with phrases like 'The correct answer is...'."*

E MCQA Datasets and Individual Evaluation Results for Chain-of-Thought Rationales

Table 14 presents accuracy scores across individual datasets evaluated on the `mnlp-stem-mcqa` benchmark using basic SFT with CoT. Accuracy consistently drops when using GPT-generated rationales in place of original ones. Notably, the GPT-based CoT for QASC and MathQA results in scores below 0.4, indicating low-quality reasoning traces.

We therefore exclude these two datasets from our final training set.

Dataset	Rationales	Accuracy	StdErr
M1	GPT-generated	0.4156	± 0.0177
OpenBookQA	Original	0.4130	± 0.0178
	GPT-generated	0.3610	± 0.0173
QASC	Original	0.3961	± 0.0176
	GPT-generated	0.3299	± 0.0170
SciQ	Original	0.4169	± 0.0178
MathQA	Original	0.3818	± 0.0175

Table 14: Datasets accuracy scores evaluated on the mnlp-stem-mcqa benchmark using basic SFT and CoT

F GPT-4o Summarization Prompt for RAG Dataset Preparation

The following prompt was used with GPT-4o to generate concise summaries of scientific texts for use in RAG dataset construction:

System Prompt	You are a STEM expert tasked with summarizing scientific texts. Please provide a concise summary of the given text.
User Prompt	Please summarize the following text concisely: <TEXT>

Table 15: Prompt template used for summarization dataset construction with GPT-4o.

G Accuracy Across Epoch Settings for MCQA Model

Epochs	Learning Rate	Accuracy
1	5×10^{-5}	0.2701
3	1×10^{-5}	0.3364
5	1×10^{-5}	0.2584

Table 16: MCQA accuracy scores from basic SFT on student-contributed data, evaluated with mnlp_stem_mcqa (batch size = 4).

H Quantization Results on All Configurations

Library	Scheme	Num Samples	Smoothing	Acc	Acc norm.
M1-only Model					
LLMComp	W8A8	512	0.1	0.4104	0.3792
	W8A8	512	0.3	0.4208	0.3844
	W8A8	512	0.5	0.4234	0.3831
	W8A8	512	0.8	0.4195	0.3818
	W8A8	512	1.0	0.4117	0.3714
	W8A8	128	0.8	0.4169	0.3805
	W8A8	1024	0.8	0.4195	0.3831
	W4A16	512	0.8	0.3662	0.3286
	W4A16	128	0.8	0.3727	0.3429
	W4A16	1024	0.8	0.3870	0.3610
BNB	W8	–	–	0.4156	0.3714
	W4	–	–	0.3260	0.3104
Best MCQA model					
LLMComp	W8A8	512	0.1	0.4247	0.3351
	W8A8	512	0.3	0.4221	0.3351
	W8A8	512	0.5	0.4195	0.3299
	W8A8	512	0.8	0.4195	0.3364
	W8A8	512	1.0	0.4091	0.3260
	W8A8	128	0.8	0.4221	0.3325
	W8A8	1024	0.8	0.4286	0.3351
	W8A8	128	0.3	0.4156	0.3286
	W8A8	1024	0.3	0.4247	0.3286
	W8A8	128	0.1	0.4156	0.3260
	W8A8	1024	0.1	0.4143	0.3312
	W4A16	512	0.8	0.3636	0.3117
	W4A16	128	0.8	0.3532	0.3143
	W4A16	1024	0.8	0.3818	0.3195
	W4A16	512	0.3	0.3597	0.2987
	W4A16	128	0.3	0.3766	0.3078
	W4A16	1024	0.3	0.4052	0.3117
	W4A16	512	0.1	0.3727	0.3156
	W4A16	128	0.1	0.3558	0.3052
	W4A16	1024	0.1	0.3831	0.3143
BNB	W8	–	–	0.4130	0.3325
	W4	–	–	0.3429	0.2831

Table 17: Accuracy and normalized accuracy of quantized MCQA models on the mnlp_stem_mcqa evaluation set, comparing different compression schemes and model variants.

I Diverse Benchmark Results of Quantized Models

Library	Config	Model	Samples	Smooth	mmlu	nlp4ed	ai2arc
LLMComp	W8A8	M1-only	1024	0.8	0.4702	0.4905	0.6957
	W4A16	M1-only	1024	0.3	0.4312	0.4523	0.6482
BNB	W4	M1-only	–	–	0.3824	0.3950	0.5953
	W8	Best	–	–	0.4759	0.4606	0.6928

Table 18: Quantized model performance across various evaluation datasets

J RAG Similarity Function and Retrieval Depth Experiments

Similarity Function Analysis Table 19 compares various similarity functions used during retrieval in the RAG pipeline. Cosine similarity consistently outperforms dot product, maximum inner product, and Jaccard similarity, particularly when applied to the `MCQA-Baseline` setup, yielding an accuracy of 0.491. This suggests that cosine similarity is better aligned with the embedding space structure used in our models.

Base Model	Similarity Function	Accuracy	Std. Dev.
`Qwen 3-0.6B`	**Cosine**	**0.350**	± 0.0479
	Dot Product	0.320	
	Max Inner Product	0.275	
	Jaccard	0.285	
`MCQA-Baseline`	**Cosine**	**0.491**	± 0.0158
	Dot Product	0.487	
	Max Inner Product	0.487	
	Jaccard	0.487	

Table 19: Performance comparison of different similarity functions over `mmlu`

Retrieval Depth Sensitivity Table 20 explores the impact of retrieval depth (i.e., the number of retrieved documents) on accuracy. Surprisingly, retrieving 10 documents yields higher accuracy than retrieving 5 or 20 documents, indicating that overly broad retrieval may introduce noise, while too narrow a scope limits context.

# Retrieved Documents	Accuracy	Std. Dev.
10	**0.7224**	± 0.0076
20	0.2816	
5	0.7130	

Table 20: Effect of retrieval depth on performance for `MCQA-Baseline` evaluated over `ai2arc`

K Failure Frequency Analysis of DPO Evaluation Examples

ID	Example (Truncated)	# Models Failed
24	Craft an intriguing opening paragraph for a fictional short story...	16
453	Rust function to sort number words like "zero" to "nine"...	16
98	Let S be the union of the set of all points inside a regular nonagon...	16
294	JavaScript function `derivative(xs)` returning polynomial derivative...	16
88	Largest angle in a convex quadrilateral with two right angles...	16
247	Go function `RightAngleTriangle(a, b, c)` to check for right triangle...	15
408	Rust function `flip_case(string)` to invert character cases...	15
11	Equiangular octagon with alternating side lengths...	15
7	Steel sphere carved from cube — find cube's volume...	15
487	JavaScript function `sameChars(s0, s1)` for set equality of characters...	15

Table 21: Top 10 most frequently failed DPO evaluation examples (reward accuracy = 0 across models).